

You are an Elite Enterprise Software Architect and UiPath Platform Engineer. I am building a hackathon project called AuraQA — a Self-Healing Enterprise Test Automation Engine.

Your task is to generate the **Master Architecture Document (MAD)**. I will be feeding your exact output directly into an autonomous coding agent (like Cursor or Windsurf) to write the actual code.

Therefore, your output must be a highly structured, strictly technical blueprint. Eliminate all fluff. Prioritize schemas, file paths, exact function signatures, and deterministic logic.

1. Project Summary

AuraQA detects broken UI test selectors caused by front-end changes, autonomously locates the shifted DOM element using computer vision/DOM analysis, rewrites the broken selector via UiPath for Coding Agents (Claude/Gemini), provisions synthetic test data via Test Data Orchestrator (TDO), re-executes the healed test, and logs a full audit report to Jira.

2. Tech Stack Constraints

- **Core Automation:** UiPath Test Cloud, Test Manager, Autopilot for Test Suite.
- **Agentic Layer:** Python 3.10+, LangGraph (for reasoning/routing), UiPath for Coding Agents CLI.
- **Integrations:** UiPath Integration Service (Jira connector), Test Data Orchestrator (TDO).
- **Target App:** React/Tailwind mock web app (intentionally shifting CSS/XPath locators).
- **Deployment:** UiPath CLI (uip).

3. Required Outputs

Generate the Master Architecture Document using the following exact structure:

Phase 1: Core Data Models & Schemas

Before writing any architecture, define the exact data structures that will be passed between UiPath and Python.

- Provide the Python TypedDict for the LangGraph State.
- Provide the JSON schema for the payload sent from UiPath upon a test failure.
- Provide the JSON schema for the payload returned by Python to UiPath (containing the healed selector and confidence score).

Phase 2: Complete Project Directory Tree

Provide a comprehensive ASCII folder tree.

- Include every single file required (e.g., main.py, graph.py, project.json, mock_app/src/App.jsx, etc.).

- Append a 1-sentence comment to each file explaining its exact technical role.

Phase 3: LangGraph Agent Specification

Define the self-healing reasoning agent. For this section, provide actual Python code stubs.

- **Nodes:** Define exact function signatures (def analyze_dom(...), def generate_selector(...)) with their inputs and outputs.
- **Edges/Routing:** Define the conditional logic (e.g., if confidence < 80%, route to Human Escalation).
- **Tools:** Define the exact tool calls the agent can use.

Phase 4: Module & Integration Contracts

Detail the exact boundary contracts between systems.

- **UiPath to Python Handoff:** How does UiPath invoke the LangGraph agent? (REST API? CLI execution?) Provide the exact command or endpoint.
- **UiPath to Jira:** Exact Integration Service connector configuration requirements.
- **UiPath to TDO:** Exact method used to provision the synthetic data.

Phase 5: The "Self-Healing" Algorithm logic

Write out a step-by-step pseudo-code or text flow of the DOM parsing algorithm. How exactly does the system know *which* new element corresponds to the broken selector? (e.g., text matching, proximity scoring, visual bounding boxes).

Phase 6: AI Coding Agent Implementation Plan

Break down the implementation into strict, sequential steps for the autonomous coding agent. Use this format:

- **Step 1: Mock App & CI Scaffolding:** (List files to create).
- **Step 2: Data Models & LangGraph Skeleton:** (List files to create).
- **Step 3: UiPath Workflows & Contracts:** (List exact sequence of UiPath Studio steps).
- **Step 4: Integration & E2E Testing.**

Phase 7: Deployment & CLI Sequence

Provide the exact, raw CLI commands (using uip) to format, pack, and deploy the UiPath components, meant to be copy-pasted into a terminal.

Output Formatting Rules

- Use deep Markdown formatting (Headers, bolding, code blocks).
- Every code block must include the file path at the top (e.g., `/// mock_app/package.json`).
- DO NOT output placeholders like `// Add business logic here`. Write out the structural code.
- DO NOT include presentation advice or business pitch information. This document is

strictly for machine ingestion and technical execution.